Assignment 1 · Part 2 — Interpolation of rotations

Exported 2026-11-06 · 36 cells

25 points

Animating a virtual human interpolates between poses, i.e. between rotations. The representation (Euler angles or quaternions) and the interpolation scheme determine whether the motion is smooth, takes the shortest path and has constant angular speed. You implement four schemes and compare them on a test object, the letter F (Lecture 02.2, slides 12–19 and 28–32).

Optional: the interactive companion `assignment_1_lab.html` (open it in a browser; tabs "Gimbal lock", "Unit circle", "Comparison") visualises the concepts of this part. Nothing in this notebook depends on it.

Instructions

Complete the code between `START OF YOUR SOLUTION` and `END OF YOUR SOLUTION` (replace every `None`) and answer the written questions in the cells headed **Answer**. A check cell follows each coding block; it compares your functions with reference values and is meant as a self-test — grading is based on your code and your written answers. Inside formulas angles are in radians (`np.deg2rad` converts degrees); rotations are counter-clockwise. Submit the notebook together with a PDF export, one submission per team.

Notation used in this part

Symbol	Meaning
$\mathbf{R}_x(\alpha), \mathbf{R}_y(\beta), \mathbf{R}_z(\gamma)$	rotations about the coordinate axes (active, counter-clockwise; lecture slides 14 and 17)
$\mathbf{R}(\alpha,\beta,\gamma) = \mathbf{R}_x(\alpha)\,\mathbf{R}_y(\beta)\,\mathbf{R}_z(\gamma)$	**Euler angles**, $\boldsymbol{\eta} = (\alpha,\beta,\gamma)^\top$ (slide 14): intrinsic rotations about x, then y', then z''. Singular ("gimbal lock") at $\beta = \pm\pi/2$: $\mathbf{R}$ then depends only on $\alpha+\gamma$ (at $+\pi/2$) or on $\alpha-\gamma$ (at $-\pi/2$)
$\mathbf{q} = (q_w, \mathbf{v}) = (q_w, q_x, q_y, q_z)^\top$	**quaternion**, scalar first. A rotation by θ about the unit axis $\boldsymbol{\omega}$ is $q_w = \cos\frac{\theta}{2},\ \mathbf{v} = \sin\frac{\theta}{2}\,\boldsymbol{\omega}$ (slide 29)
$\mathbf{p}\circ\mathbf{q} = \big(p_w q_w - \mathbf{v}_p\cdot\mathbf{v}_q,\ \ p_w\mathbf{v}_q + q_w\mathbf{v}_p + \mathbf{v}_p\times\mathbf{v}_q\big)$	quaternion product of $\mathbf{p} = (p_w,\mathbf{v}_p)$ and $\mathbf{q} = (q_w,\mathbf{v}_q)$ — concatenates rotations (slide 30, where it is written $\mathbf{q}_1 \circ \mathbf{q}_2$)
$\bar{\mathbf{q}} = (q_w, -\mathbf{v}),\ \ \mathbf{q}^{-1} = \bar{\mathbf{q}}/\lVert\mathbf{q}\rVert^2$	conjugate (the bar of the slides; not the homogeneous bar of Part 1) and inverse
$\ln\mathbf{q} = \big(\ln\lVert\mathbf{q}\rVert,\ \frac{\mathbf{v}}{\lVert\mathbf{v}\rVert}\arccos\frac{q_w}{\lVert\mathbf{q}\rVert}\big)$	logarithm
$\exp(q_w,\mathbf{v}) = e^{q_w}\big(\cos\lVert\mathbf{v}\rVert,\ \frac{\mathbf{v}}{\lVert\mathbf{v}\rVert}\sin\lVert\mathbf{v}\rVert\big)$	exponential
$\mathbf{q}^t = \exp(t\ln\mathbf{q})$	real power, $t\in\mathbb{R}$
$t_k = \dfrac{k}{n-1},\ k = 0,\dots,n-1$	parameter of the k-th of n frames — **both end points are included**

A *unit* quaternion ($\lVert\mathbf{q}\rVert = 1$) is a rotation. The matrix of $\mathbf{x}\mapsto\mathbf{q}\circ\mathbf{x}\circ\bar{\mathbf{q}}$ is provided as

`quat_to_matrix(q)`; for a **non-unit** $\mathbf{q}$ it equals $\|\mathbf{q}\|^2$ times a rotation — the object is scaled. We deliberately do **not** normalise inside it.

Convention: everywhere in this assignment rotations are **active** and counter-clockwise, as on slides 9, 14 and 17.

```
#@title Setup — run this cell first
import os, sys, zipfile

# The helper package `vh_a1` (plots, given functions, self-checks) ships as vh_a1.zip next to this notebook.
if not os.path.isdir("vh_a1"):
    if not os.path.exists("vh_a1.zip") and "google.colab" in sys.modules:
        from google.colab import files
        print("Select the file vh_a1.zip (part of the assignment material):")
        uploaded = files.upload()
        if uploaded:
            os.replace(next(iter(uploaded)), "vh_a1.zip")
    if not os.path.exists("vh_a1.zip"):
        raise RuntimeError("Package not found: put the folder vh_a1/ or the file vh_a1.zip next to this notebook and run this cell again.")
    zipfile.ZipFile("vh_a1.zip").extractall(".")

%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
from vh_a1 import params
from vh_a1.common import Checker, progress, ready
np.set_printoptions(precision=4, suppress=True)
```

```
from vh_a1 import part2 as G
from vh_a1.part2 import (euler_to_matrix, euler_to_quat, quat_to_matrix, quat_conj, quat_norm, quat_inv,
                         interpolate_sequence, plot_metrics, filmstrip, swept_angle_deg, angular_step_deg)

# The four test sequences: waypoints (alpha, beta, gamma) in degrees, lecture convention R = Rx(alpha) Ry(beta) Rz(gamma)
seq0 = [(0, 0, -125), (0, 0, 125)]
seq1 = [(40, 40, 0), (40, 90, 0), (-140, 50, 180), (40, 40, 0)]
seq2 = [(0, 0, 0), (0, 0, 60), (180, 0, 60), (180, 90, 0), (0, 0, 0)]
seq3 = [(-170, 0, 140), (-160, -40, -140), (-170, 0, 140)]
SEQ = params.SEQUENCES_DEG
```

The test object and the test sequences

Everything is shown on the letter F. A *sequence* is a list of waypoints (Euler angles in degrees). Between each pair of neighbouring waypoints an interpolation produces n frames; `interpolate_sequence(waypoints, func, n, kind)` (given) applies your function to every pair and stacks the resulting 3×3 matrices. Every segment gets the **same** number of frames, so the level of the speed curve changes at the waypoints (the segments differ in length); compare its *shape* within a segment.

- `seq0` — one axis only: γ from -125° to 125°.
- `seq1` — several angles change at once and the path goes through $\beta = 90^\circ$.
- `seq2` — contains two segments in which the object turns by exactly 180°.
- `seq3` — two orientations that are only about 85° apart, but written with γ values that are 280° apart (140° and -140°).

Task 2.1 · Naive interpolation of Euler angles (5 points)

The simplest way to move from start to end in n frames is linear interpolation,

$$\mathrm{value}(t_k) = (1 - t_k)\,\mathrm{start} + t_k\,\mathrm{end}, \qquad t_k = \tfrac{k}{n-1}. \tag{1}$$

Applied to the Euler angles $\boldsymbol{\eta} = (\alpha, \beta, \gamma)^\top$ this gives

$$\boldsymbol{\eta}(t_k) = (1 - t_k)\,\boldsymbol{\eta}_0 + t_k\,\boldsymbol{\eta}_1, \qquad \mathbf{R}_k = \mathbf{R}\big(\boldsymbol{\eta}(t_k)\big). \tag{2}$$

Task. Implement `euler_lerp(angles0, angles1, n)` : return the $n \times 3$ array of interpolated angles (radians). The conversion to matrices is done for you.

```
#### START OF YOUR SOLUTION ####
def euler_lerp(angles0, angles1, n):
    t = np.linspace(0, 1, n)
    return (1 - t[:, None]) * angles0 + t[:, None] * angles1
#### END OF YOUR SOLUTION ####
```

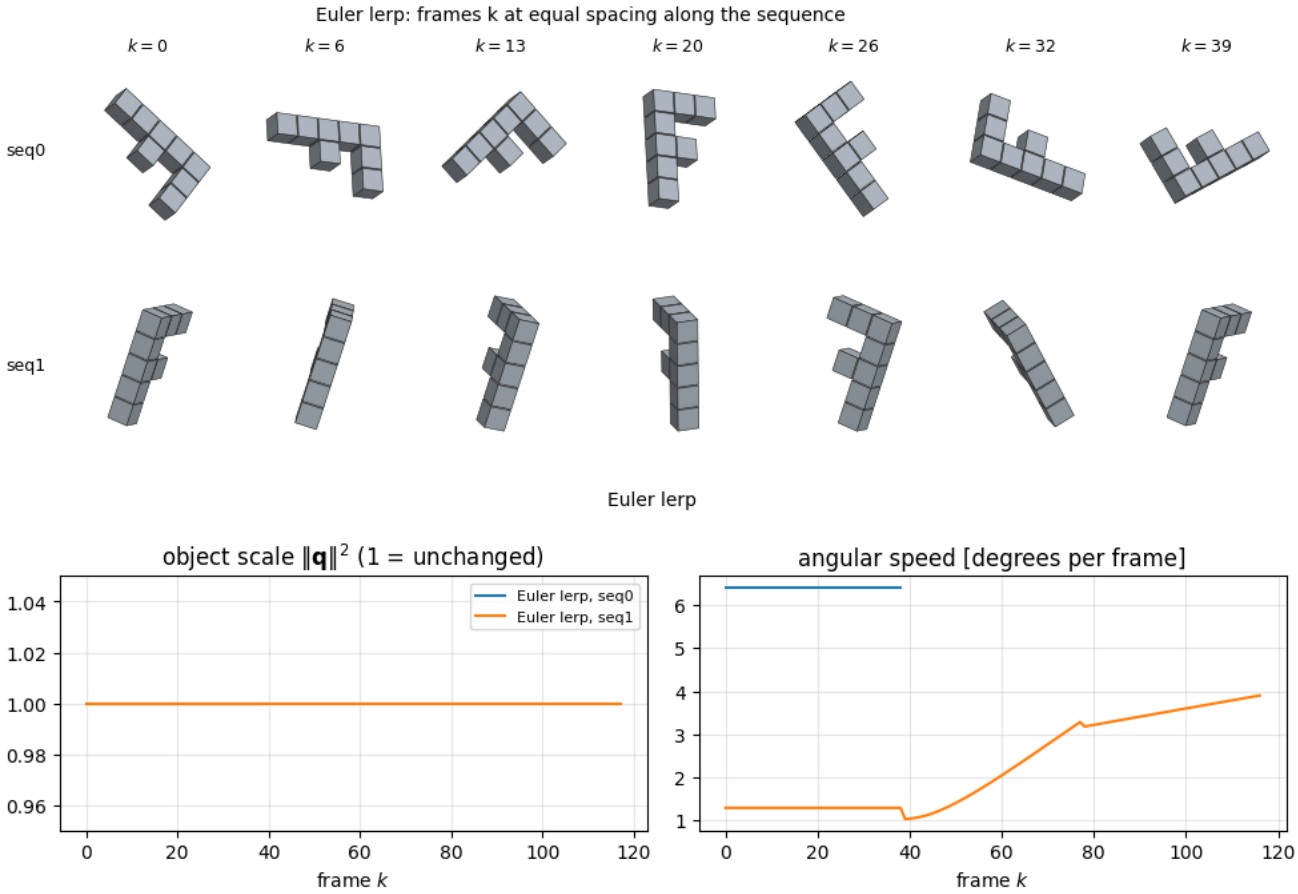
```
chk = Checker("Part 2 · Task 2.1")
A, B = np.array([0.1, 0.2, 0.3]), np.array([1.1, -0.4, 0.9])
chk.close("euler_lerp(A, B, 5)", lambda: euler_lerp(A, B, 5), [[0.1, 0.2, 0.3], [0.35,
0.05, 0.45], [0.6, -0.1, 0.6], [0.85, -0.25, 0.75], [1.1, -0.4, 0.9]],
          hint="n frames, t_k = k/(n-1): the first row is A and the LAST row is B")
chk.test("the end point is reached (last row = B)", lambda: None if euler_lerp(A, B, 7) is
None else bool(np.allclose(euler_lerp(A, B, 7)[-1], B)))
chk.close("no wrapping of the angles: γ from −2.2 to 2.2 (more than π apart) is still
linear", lambda: euler_lerp(np.array([0, 0, -2.2]), np.array([0, 0, 2.2]), 5),
          [[0, 0, -2.2], [0, 0, -1.1], [0, 0, 0], [0, 0, 1.1], [0, 0, 2.2]], hint="Eq. (2)
interpolates the three numbers linearly; do not map the difference to (−π, π]")
```

passedeuler_lerp(A, B, 5)

passedthe end point is reached (last row = B)

passedno wrapping of the angles: γ from −2.2 to 2.2 (more than π apart) is still linear

```
euler_frames = {k: interpolate_sequence(SEQ[k], euler_lerp, params.N_FRAMES, "euler") for
k in ("seq0", "seq1")}
if ready(*euler_frames.values()):
    filmstrip({"seq0": euler_frames["seq0"], "seq1": euler_frames["seq1"]}, title="Euler
lerp: frames k at equal spacing along the sequence")
    plot_metrics({"Euler lerp, seq0": euler_frames["seq0"], "Euler lerp, seq1":
euler_frames["seq1"]}, "Euler lerp")
```



Answer — describe `seq0` and `seq1`: is the path the shortest one, does the object turn without jerks within a segment, is the angular speed constant within a segment?

1. `seq0`: *The object turn without jerks and the angular speed is constant but it is not the shortest path*
2. `seq1`: *The rotation has some jerks and the angluar speed is not constant or travel the shortest path*
3. Why? (What is wrong with interpolating the three angles independently?) *Interpolating the angles independently means it interpolates the numbers not the actual rotation so it does not get the shortest path*

Task 2.2 · Naive interpolation of quaternions (5 points)

Quaternions avoid gimbal lock. Apply (1) to the four components,

$$\mathbf{q}(t_k) = (1 - t_k)\,\mathbf{q}_0 + t_k\,\mathbf{q}_1. \tag{3}$$

Task. Implement `quat_lerp(q0, q1, n)` (an $n \times 4$ array, scalar first). **Do not normalise.**

```
#### START OF YOUR SOLUTION ####
def quat_lerp(q0, q1, n):
    t = np.linspace(0, 1, n)
    return (1 - t[:, None]) * q0 + t[:, None] * q1
#### END OF YOUR SOLUTION ####
```

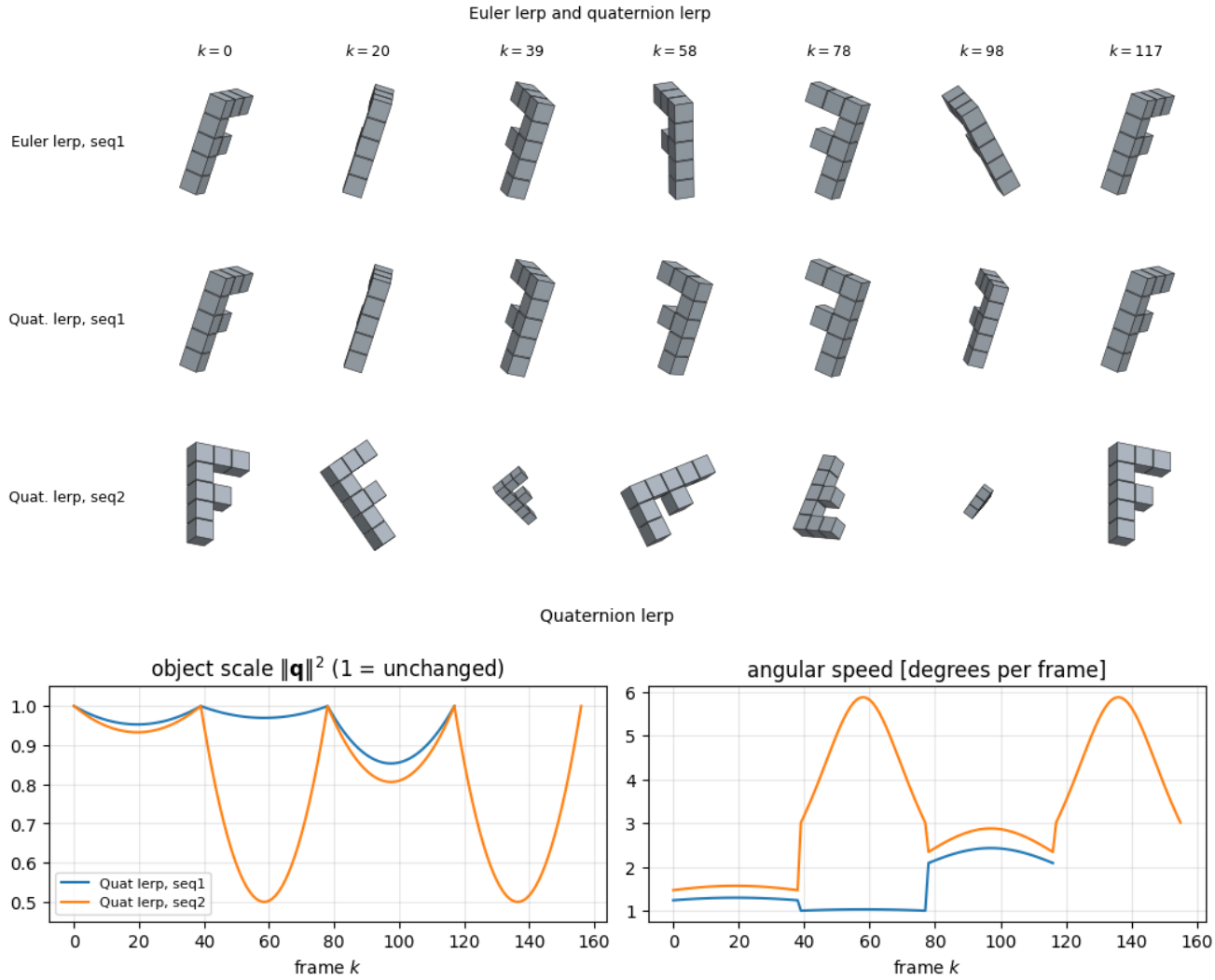
```
chk = Checker("Part 2 · Task 2.2")
qA, qB = euler_to_quat(A), euler_to_quat(B)
chk.close("quat_lerp(qA, qB, 5)", lambda: quat_lerp(qA, qB, 5), [[0.981856, 0.064071,
0.091158, 0.153439], [0.935772, 0.144954, -0.025464, 0.18256], [0.889688, 0.225836,
-0.142085, 0.211681], [0.843603, 0.306718, -0.258707, 0.240802], [0.797519, 0.3876,
-0.375328, 0.269923]], hint="plain (1−t) q0 + t q1 on the 4 components")
chk.test("not normalised: the middle frame is shorter than 1", lambda: None if
quat_lerp(qA, qB, 3) is None else bool(np.linalg.norm(quat_lerp(qA, qB, 3)[1]) < 1 -
1e-6),
         hint="Eq. (3) has no division by the norm — that is Task 2.3")
qN0, qN1 = np.array([1.0, 0.0, 0.0, 0.0]), np.array([-0.6, 0.8, 0.0, 0.0])              #
q0·q1 = −0.6
chk.close("quat_lerp with q0·q1 = −0.6 < 0 is still the plain formula (no sign flip)",
lambda: quat_lerp(qN0, qN1, 5), [[1.0, 0.0, 0.0, 0.0], [0.6, 0.2, 0.0, 0.0], [0.2, 0.4,
0.0, 0.0], [-0.2, 0.6, 0.0, 0.0], [-0.6, 0.8, 0.0, 0.0]], hint="Eq. (3) as it stands; the
sign question comes in Task 2.4(c)")
```

passedquat_lerp(qA, qB, 5)

passednot normalised: the middle frame is shorter than 1

passedquat_lerp with q0·q1 = −0.6 < 0 is still the plain formula (no sign flip)

```
quat_frames = {k: interpolate_sequence(SEQ[k], quat_lerp, params.N_FRAMES, "quat") for k
in ("seq1", "seq2")}
if ready(*quat_frames.values()) and ready(*euler_frames.values()):
    filmstrip({"Euler lerp, seq1": euler_frames["seq1"], "Quat. lerp, seq1":
quat_frames["seq1"], "Quat. lerp, seq2": quat_frames["seq2"]}, title="Euler lerp and
quaternion lerp")
    plot_metrics({"Quat lerp, seq1": quat_frames["seq1"], "Quat lerp, seq2":
quat_frames["seq2"]}, "Quaternion lerp")
```



Answer — compare with Task 2.1 on `seq1`, and describe `seq2`.

1. `seq1`: differences to Euler-lerp? *The angular speed in Quaternion-lerp is a bit more stable than in Eurler-lerp but the scale of the objects changes. Both interpolation mehtods are jerky*
2. `seq2`: which artefact appears, and where is it strongest? The object gets scaled down, it it most noticeable in $k = 98$ and $k = 39$
3. Explain the artefact: what is $\|\mathbf{q}(t)\|$ along a segment and what does `quat_to_matrix` do with it? $\|\mathbf{q}\|$ *is the length of a segment. For example if we are interpolating between* $q_0 = (1, 0, 0, 0)$ *and* $q_1 = (0, 1, 0, 0)$ *in the halfwaypoint* $q(0.5)$ *the length would be* $\sqrt{(0.5)^2 + (0.5)^2} = \sqrt{0.5}$ *which is no longer a unit quaternion.* `quat_to_matrix` *scales the object by* $\|\mathbf{q}\|^2$ *when the quaternion is not a unit quaternion*

Task 2.3 · Nlerp: normalised linear interpolation (5 points)

The size artefact comes from non-unit quaternions. Normalise (3) to stay on the unit sphere:

$$\mathbf{q}(t_k) = \frac{(1-t_k)\,\mathbf{q}_0 + t_k\,\mathbf{q}_1}{\left\|(1-t_k)\,\mathbf{q}_0 + t_k\,\mathbf{q}_1\right\|}. \tag{4}$$

Task. Implement `nlerp(q0, q1, n)`.

```
#### START OF YOUR SOLUTION ####
def nlerp(q0, q1, n):
    q = quat_lerp(q0, q1, n)
    return q / np.linalg.norm(q, axis=1, keepdims=True)
#### END OF YOUR SOLUTION ####
```

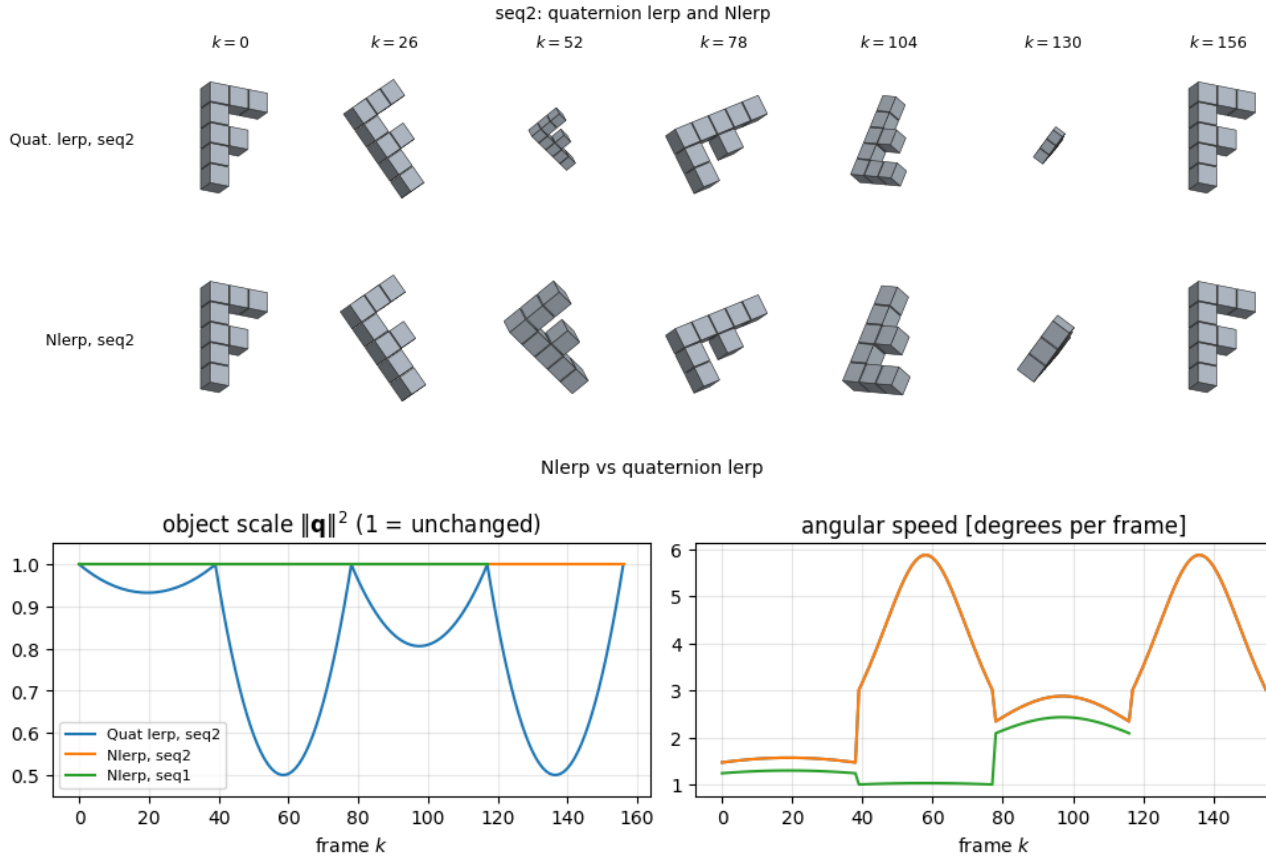
```
chk = Checker("Part 2 · Task 2.3")
chk.close("nlerp(qA, qB, 5)", lambda: nlerp(qA, qB, 5), [[0.981856, 0.064071, 0.091158,
0.153439], [0.970008, 0.150257, -0.026395, 0.189239], [0.933908, 0.23706, -0.149147,
0.222202], [0.874467, 0.317939, -0.268172, 0.249612], [0.797519, 0.3876, -0.375328,
0.269923]], hint="divide each row by its norm")
chk.test("every frame is a unit quaternion", lambda: None if nlerp(qA, qB, 9) is None else
bool(np.allclose(np.linalg.norm(nlerp(qA, qB, 9), axis=1), 1)))
chk.close("nlerp with q0·q1 = −0.6 < 0 is still the plain formula (no sign flip)", lambda:
nlerp(qN0, qN1, 5), [[1.0, 0.0, 0.0, 0.0], [0.948683, 0.316228, 0.0, 0.0], [0.447214,
0.894427, 0.0, 0.0], [-0.316228, 0.948683, 0.0, 0.0], [-0.6, 0.8, 0.0, 0.0]], hint="Eq.
(4) as it stands; the sign question comes in Task 2.4(c)")
```

passednlerp(qA, qB, 5)

passedevery frame is a unit quaternion

passednlerp with q0·q1 = −0.6 < 0 is still the plain formula (no sign flip)

```
nlerp_frames = {k: interpolate_sequence(SEQ[k], nlerp, params.N_FRAMES, "quat") for k in
("seq1", "seq2")}
if ready(*nlerp_frames.values()) and ready(*quat_frames.values()):
    filmstrip({"Quat. lerp, seq2": quat_frames["seq2"], "Nlerp, seq2":
nlerp_frames["seq2"]}, title="seq2: quaternion lerp and Nlerp")
    plot_metrics({"Quat lerp, seq2": quat_frames["seq2"], "Nlerp, seq2":
nlerp_frames["seq2"], "Nlerp, seq1": nlerp_frames["seq1"]}, "Nlerp vs quaternion lerp")
```



Answer

1. What does Nlerp fix compared to Task 2.2? Which artefact remains visible in the speed plot? *Nlerp fixes the scaling problem but the angular speed remains not constant*
2. Is the angular speed constant? Where is the F fastest? *The angular speed is not constant, it is fastest between $k = 52$ and $k = 78$ and between $k = 130$ and $k = 156$*

Task 2.4 · Slerp: spherical linear interpolation (10 points)

Slerp moves along the great circle between the unit quaternions with **constant angular velocity**:

$$\mathrm{Slerp}(\mathbf{q}_0, \mathbf{q}_1, t) = \mathbf{q}_0 \circ \left(\mathbf{q}_0^{-1} \circ \mathbf{q}_1\right)^t, \qquad t \in [0, 1]. \tag{5}$$

The power is defined through $\exp$ and $\ln$ (see the notation box): $\mathbf{q}^t = \exp(t \ln \mathbf{q})$.

(a) Quaternion algebra (4 points). Implement `quat_mul` ($\circ$), `quat_log` and `quat_exp`. The degenerate case $\mathbf{v} = \mathbf{0}$ is already handled for you; you write the general branch.

(b) Slerp (3 points). Implement `slerp(q0, q1, n)` *exactly* as in (5).

```
##### START OF YOUR SOLUTION #####
```

```
def quat_mul(p, q):
    pw, pv = p[0], p[1:]
    qw, qv = q[0], q[1:]
    return np.concatenate((
        [pw * qw - pv @ qv],
        pw * qv + qw * pv + np.cross(pv, qv)
        ))

def quat_log(q):
    w, v = q[0], q[1:]
    nq, nv = np.linalg.norm(q), np.linalg.norm(v)
    if nv < 1e-12:                                  # (given) no vector part: ln q = (ln||
q||, 0)
        return np.array([np.log(nq), 0.0, 0.0, 0.0])
    return np.concatenate((
        [np.log(nq)],
        v / nv * np.arccos(w / nq)))

def quat_exp(q):
    w, v = q[0], q[1:]
    nv = np.linalg.norm(v)
    if nv < 1e-12:                                  # (given) no vector part: exp q =
(e^w, 0)
        return np.array([np.exp(w), 0.0, 0.0, 0.0])
    return np.concatenate((
        [np.exp(w) * np.cos(nv)],
        np.exp(w) * v / nv * np.sin(nv)
    ))

def quat_pow(q, t):                                 # (given) q^t = exp(t ln q)
    return quat_exp(t * quat_log(q))

def slerp(q0, q1, n):
    t = np.linspace(0, 1, n)
    q0_inv = quat_inv(q0)
    relative = quat_mul(q0_inv, q1)

    return np.array([
        quat_mul(q0, quat_pow(relative, ti))
        for ti in t
    ])
#### END OF YOUR SOLUTION ####
```

```
chk = Checker("Part 2 · Task 2.4a quaternion algebra")
pT, qT = np.array([0.8, 0.1, -0.3, 0.5]), np.array([-0.2, 0.6, 0.4, 0.7])
chk.close("p ∘ q", lambda: quat_mul(pT, qT), [-0.45, 0.05, 0.61, 0.68], hint="(p_w q_w −
p_v·q_v,  p_w q_v + q_w p_v + p_v × q_v)")
chk.close("i ∘ j = k", lambda: quat_mul(np.array([0, 1, 0, 0.]), np.array([0, 0, 1, 0.])),
[0, 0, 0, 1])
chk.close("j ∘ i = −k (not commutative)", lambda: quat_mul(np.array([0, 0, 1, 0.]),
np.array([0, 1, 0, 0.])), [0, 0, 0, -1])
chk.close("q ∘ conj(q) = (‖q‖², 0, 0, 0)", lambda: quat_mul(qT, quat_conj(qT)), [float(qT
@ qT), 0, 0, 0], atol=1e-9)
uT, nT = np.array([0.896804, 0.038101, -0.2817, 0.339022]), np.array([1.5, 0.2, -0.4,
0.3])
chk.close("ln of a unit quaternion", lambda: quat_log(uT), [0.0, 0.039468, -0.291809,
0.351188])
chk.close("ln of a non-unit quaternion", lambda: quat_log(nT), [0.466082, 0.128011,
-0.256022, 0.192016], hint="the scalar part is ln‖q‖")
chk.close("exp(ln q) = q", lambda: quat_exp(quat_log(nT)), nT, atol=1e-9)
chk.close("exp of a pure quaternion (0, 0, 0, π/2) = (0, 0, 0, 1)", lambda:
quat_exp(np.array([0, 0, 0, np.pi / 2])), [0, 0, 0, 1], atol=1e-9)
```

passedp ∘ q

passedi ∘ j = k

passedj ∘ i = −k (not commutative)

passedq ∘ conj(q) = (‖q‖², 0, 0, 0)

passedln of a unit quaternion

passedln of a non-unit quaternion

passedexp(ln q) = q

passedexp of a pure quaternion (0, 0, 0, π/2) = (0, 0, 0, 1)

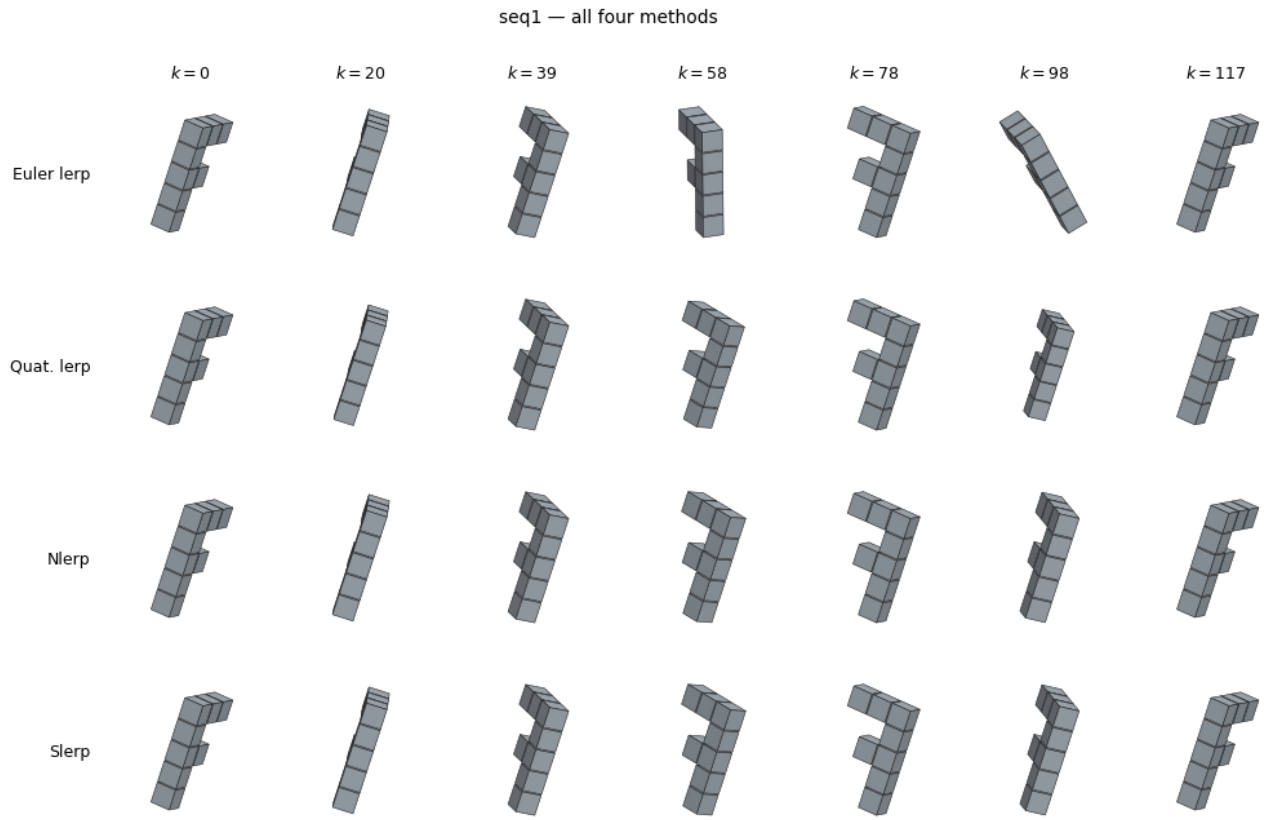
```
chk = Checker("Part 2 · Task 2.4b slerp")
q0s0, q1s0 = euler_to_quat(np.deg2rad(SEQ["seq0"][0])),
euler_to_quat(np.deg2rad(SEQ["seq0"][1]))
q0s1, q1s1 = euler_to_quat(np.deg2rad(SEQ["seq1"][0])),
euler_to_quat(np.deg2rad(SEQ["seq1"][1]))
chk.close("slerp on the first pair of seq1, 5 frames", lambda: slerp(q0s1, q1s1, 5),
[[0.883022, 0.321394, 0.321394, 0.116978], [0.842785, 0.306749, 0.415615, 0.151272],
[0.792529, 0.288457, 0.504896, 0.183767], [0.732852, 0.266736, 0.588176, 0.214078],
[0.664463, 0.241845, 0.664463, 0.241845]])
chk.close("slerp on seq0 exactly as in Eq. (5), 5 frames", lambda: slerp(q0s0, q1s0, 5),
[[0.461749, 0.0, 0.0, -0.887011], [0.854912, 0.0, 0.0, -0.518773], [1.0, 0.0, 0.0, 0.0],
[0.854912, 0.0, 0.0, 0.518773], [0.461749, 0.0, 0.0, 0.887011]],
          hint="implement the formula as it stands; do not flip any signs yet (see part
(c)); if only this check fails, look at your `quat_log` as well")
chk.test("constant angular speed: all steps equal", lambda: None if slerp(q0s1, q1s1, 30)
is None else
         (lambda s: (bool(s.std() < 1e-6 * max(s.mean(), 1e-12) + 1e-9), f"steps
{s.min():.4f}..{s.max():.4f} deg"))(angular_step_deg(np.array([quat_to_matrix(q) for q in
slerp(q0s1, q1s1, 30)]))))
```

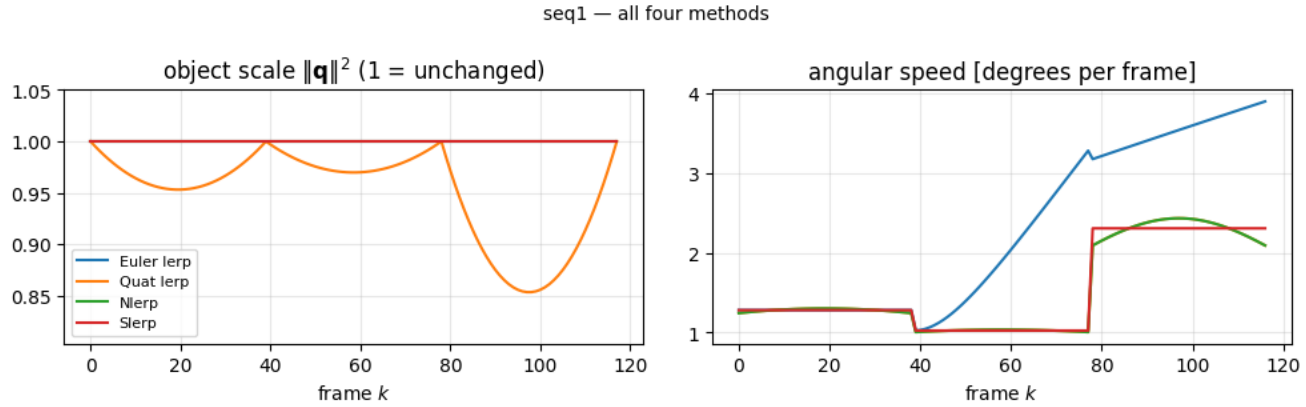
passedslerp on the first pair of seq1, 5 frames

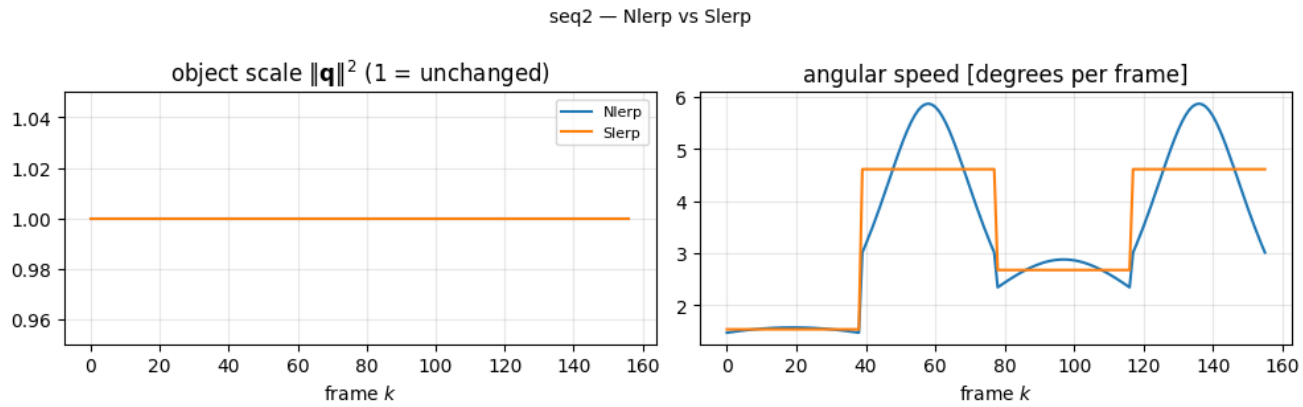
passedslerp on seq0 exactly as in Eq. (5), 5 frames

passedconstant angular speed: all steps equal

```
slerp_frames = {k: interpolate_sequence(SEQ[k], slerp, params.N_FRAMES, "quat") for k in
("seq1", "seq2", "seq3")}
if ready(*slerp_frames.values()) and ready(*nlerp_frames.values()) and
ready(*quat_frames.values()) and ready(*euler_frames.values()):
    filmstrip({"Euler lerp": euler_frames["seq1"], "Quat. lerp": quat_frames["seq1"],
"Nlerp": nlerp_frames["seq1"], "Slerp": slerp_frames["seq1"]}, title="seq1 — all four
methods")
    plot_metrics({"Euler lerp": euler_frames["seq1"], "Quat lerp": quat_frames["seq1"],
"Nlerp": nlerp_frames["seq1"], "Slerp": slerp_frames["seq1"]}, "seq1 — all four methods")
    plot_metrics({"Nlerp": nlerp_frames["seq2"], "Slerp": slerp_frames["seq2"]}, "seq2 —
Nlerp vs Slerp")
```







(c) Is Slerp the *shortest* path? (3 points). Slerp follows a great circle of the unit sphere; whether this is also the shortest rotation between the two orientations is examined on `seq0` and `seq3`.

A rotation has **two** unit quaternions, $\mathbf{q}$ and $-\mathbf{q}$ (the unit quaternions *double cover* the rotations). Eq. (5) interpolates towards the one you give it — but is that the *closer* one?

```
#### START OF YOUR SOLUTION ####
q = np.array([0.5, 0.5, 0.5, 0.5])
same_rotation = np.allclose(quat_to_matrix(q), quat_to_matrix(-q))

# seq0: the end points as quaternions
q_a, q_b = euler_to_quat(np.deg2rad(SEQ["seq0"][0])), euler_to_quat(np.deg2rad(SEQ["seq0"]
[1]))
dot_seq0 = q_a @ q_b
angle_written_deg = np.rad2deg(2 * np.arccos(dot_seq0))
angle_shortest_deg = np.rad2deg(2 * np.arccos(np.abs(dot_seq0)))
# TODO: smallest rotation angle (deg) between the two orientations
#### END OF YOUR SOLUTION ####
```

```
chk = Checker("Part 2 · Task 2.4c double cover")
chk.test("q and -q give the same matrix", lambda: None if same_rotation is None else
bool(same_rotation) is True)
chk.close("q_a · q_b for seq0", lambda: dot_seq0, -0.573576, atol=1e-5)
chk.close("angle swept by Slerp as written [deg]", lambda: angle_written_deg, 250.0,
atol=0.05)
chk.close("shortest rotation angle [deg]", lambda: angle_shortest_deg, 110.0, atol=0.05,
          hint="the angle between two rotations is 2·arccos(|q_a · q_b|)")
```

passedq and −q give the same matrix

passedq_a · q_b for seq0

passedangle swept by Slerp as written [deg]

passedshortest rotation angle [deg]

Now repair it. Implement `slerp_shortest(q0, q1, n)`: Slerp that always takes the shorter of the two possible paths (use what you found).

```
#### START OF YOUR SOLUTION ####
def slerp_shortest(q0, q1, n):
    if np.dot(q0, q1) < 0:
        q1 = -q1

    return slerp(q0, q1, n)
#### END OF YOUR SOLUTION ####
```

```
chk = Checker("Part 2 · Task 2.4c slerp_shortest")
q0s3, q1s3 = euler_to_quat(np.deg2rad(SEQ["seq3"][0])),
euler_to_quat(np.deg2rad(SEQ["seq3"][1]))
def _sw(q0, q1): return swept_angle_deg(np.array([quat_to_matrix(q) for q in
slerp_shortest(q0, q1, 60)]))
chk.close("seq0 pair: swept angle = 110°", lambda: _sw(q0s0, q1s0), 110.0, atol=0.5)
chk.close("seq3 pair: swept angle = 85.2°", lambda: _sw(q0s3, q1s3), 85.2, atol=0.5)
chk.test("giving −q1 instead of q1 changes nothing (same rotations)",
         lambda: None if slerp_shortest(q0s0, q1s0, 9) is None else
bool(np.allclose([quat_to_matrix(q) for q in slerp_shortest(q0s0, q1s0, 9)],
[quat_to_matrix(q) for q in slerp_shortest(q0s0, -q1s0, 9)], atol=1e-9)))
chk.test("when no flip is needed it equals slerp (seq1)", lambda: None if
slerp_shortest(q0s1, q1s1, 9) is None else bool(np.allclose(slerp_shortest(q0s1, q1s1, 9),
slerp(q0s1, q1s1, 9))))
cI = np.array([1.0, 0, 0, 0])
chk.close("q0·q1 = +0.26 (a rotation by 150°): no sign change", lambda: slerp_shortest(cI,
np.array([0.258819, 0.0, 0.0, 0.965926]), 5), [[1.0, 0.0, 0.0, 0.0], [0.94693, 0.0, 0.0,
0.321439], [0.793353, 0.0, 0.0, 0.608761], [0.55557, 0.0, 0.0, 0.83147], [0.258819, 0.0,
0.0, 0.965926]], hint="flip q1 exactly when q0·q1 < 0 — not for positive dot products")
chk.close("q0·q1 = −0.17 (200° as written): the sign IS changed (160° the other way)",
lambda: slerp_shortest(cI, np.array([-0.173648, 0.0, 0.0, 0.984808]), 5), [[1.0, 0.0, 0.0,
0.0], [0.939693, 0.0, 0.0, -0.34202], [0.766044, 0.0, 0.0, -0.642788], [0.5, 0.0, 0.0,
-0.866025], [0.173648, 0.0, 0.0, -0.984808]], hint="flip q1 exactly when q0·q1 < 0 — also
when the dot product is only slightly negative")
```

passedseq0 pair: swept angle = 110°

passedseq3 pair: swept angle = 85.2°

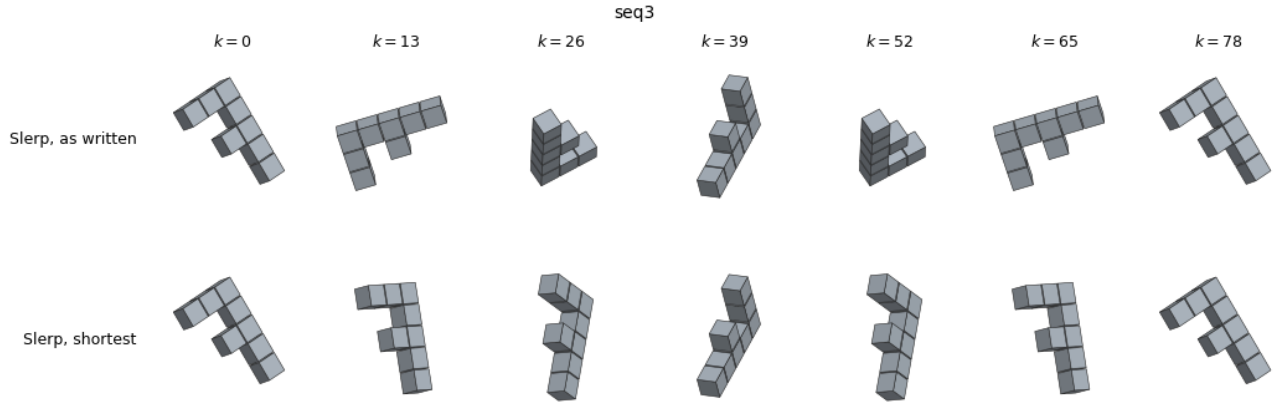
passedgiving −q1 instead of q1 changes nothing (same rotations)

passedwhen no flip is needed it equals slerp (seq1)

passedq0·q1 = +0.26 (a rotation by 150°): no sign change

passedq0·q1 = −0.17 (200° as written): the sign IS changed (160° the other way)

```
fixed = {k: interpolate_sequence(SEQ[k], slerp_shortest, params.N_FRAMES, "quat") for k in
("seq0", "seq3")}
written = {k: interpolate_sequence(SEQ[k], slerp, params.N_FRAMES, "quat") for k in
("seq0", "seq3")}
if ready(*fixed.values()) and ready(*written.values()):
    filmstrip({"Slerp, as written": written["seq3"], "Slerp, shortest": fixed["seq3"]},
title="seq3")
    for k in ("seq0", "seq3"):
        print(f"{k}: swept angle  as written {swept_angle_deg(written[k]):6.1f}°
shortest {swept_angle_deg(fixed[k]):6.1f}°")
```




```
seq0: swept angle  as written  250.0°   shortest  110.0°
seq3: swept angle  as written  549.7°   shortest  170.3°
```

Answer

1. What does Slerp do better than Nlerp? Compare the speed curves on `seq1` and on the 180° segments of `seq2`. Is the speed constant?
   Slerp is better at maintaining a constant angular speed
2. Why do $\mathbf{q}$ and $-\mathbf{q}$ give the same rotation? Which of the two does Eq. (5) interpolate towards, and when is that the longer way? How large is the detour (angle swept as written minus shortest angle) for the pair of `seq0` and for the first pair of `seq3`?
   A rotation has two equivalent unit quaternions that produce the same matrix. The chosen direction informs the interpolation so it can travel longer even if there is a shorter path. Detour: `seq0` *- 140º,* `seq3` *- 379.4º*
3. State the rule you implemented in `slerp_shortest` and why it works (what happens to the arc length on the unit sphere?).
   In `slerp_shortest` *we calculate the dot product of two quaternions, if the result is negative we negate the second quaternion before implementing slerp. Since the negative quaternion represent the same rotation, negating the second selects the shorter arc*

Summary of the checks

```
progress()
```

checks	passed	failed	not implemented
Part 2 · Task 2.1	3	0	0
Part 2 · Task 2.2	3	0	0
Part 2 · Task 2.3	3	0	0
Part 2 · Task 2.4a quaternion algebra	8	0	0
Part 2 · Task 2.4b slerp	3	0	0
Part 2 · Task 2.4c double cover	4	0	0
Part 2 · Task 2.4c slerp_shortest	6	0	0


```
{'Part 2 · Task 2.1': {'euler_lerp(A, B, 5)': 'pass',
  'the end point is reached (last row = B)': 'pass',
  'no wrapping of the angles: γ from −2.2 to 2.2 (more than π apart) is still linear':
'pass'},
 'Part 2 · Task 2.2': {'quat_lerp(qA, qB, 5)': 'pass',
  'not normalised: the middle frame is shorter than 1': 'pass',
  'quat_lerp with q0·q1 = −0.6 < 0 is still the plain formula (no sign flip)': 'pass'},
 'Part 2 · Task 2.3': {'nlerp(qA, qB, 5)': 'pass',
  'every frame is a unit quaternion': 'pass',
  'nlerp with q0·q1 = −0.6 < 0 is still the plain formula (no sign flip)': 'pass'},
 'Part 2 · Task 2.4a quaternion algebra': {'p ∘ q': 'pass',
  'i ∘ j = k': 'pass',
  'j ∘ i = −k (not commutative)': 'pass',
  'q ∘ conj(q) = (‖q‖², 0, 0, 0)': 'pass',
  'ln of a unit quaternion': 'pass',
  'ln of a non-unit quaternion': 'pass',
  'exp(ln q) = q': 'pass',
  'exp of a pure quaternion (0, 0, 0, π/2) = (0, 0, 0, 1)': 'pass'},
 'Part 2 · Task 2.4b slerp': {'slerp on the first pair of seq1, 5 frames': 'pass',
  'slerp on seq0 exactly as in Eq. (5), 5 frames': 'pass',
  'constant angular speed: all steps equal': 'pass'},
 'Part 2 · Task 2.4c double cover': {'q and −q give the same matrix': 'pass',
  'q_a · q_b for seq0': 'pass',
  'angle swept by Slerp as written [deg]': 'pass',
  'shortest rotation angle [deg]': 'pass'},
 'Part 2 · Task 2.4c slerp_shortest': {'seq0 pair: swept angle = 110°': 'pass',
  'seq3 pair: swept angle = 85.2°': 'pass',
  'giving −q1 instead of q1 changes nothing (same rotations)': 'pass',
  'when no flip is needed it equals slerp (seq1)': 'pass',
  'q0·q1 = +0.26 (a rotation by 150°): no sign change': 'pass',
  'q0·q1 = −0.17 (200° as written): the sign IS changed (160° the other way)': 'pass'}}
```